

Trabalho 03

Aluno Eduardo Lopes Jonker
Linguagem : Python

Programa : MAIN (TERMO_SOLVER) termo_solver.py

```
#!/usr/bin/env python3
import os
from typing import List, Optional

class TrieNode:
    """Representa um nó na árvore Trie."""
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    """Implementação da estrutura de dados Trie para autocompletar."""
    def __init__(self):
        self.root = TrieNode()

    def inserir(self, palavra: str) -> None:
        """Insere uma palavra na árvore."""
        nodo = self.root
        for char in palavra.lower():
            if char not in nodo.children:
                nodo.children[char] = TrieNode()
            nodo = nodo.children[char]
        nodo.is_end_of_word = True

    def buscar_prefixo(self, prefixo: str) -> Optional[TrieNode]:
        """Navega na árvore até o final do prefixo fornecido."""
        nodo = self.root
        for char in prefixo.lower():
            if char not in nodo.children:
                return None # Prefixo não existe
            nodo = nodo.children[char]
        return nodo

    def listar_palavras(self, nodo: TrieNode, prefixo: str, sugestoes: List[str]) -> None:
        """Busca recursiva por todas as palavras a partir de um nó."""
        if nodo.is_end_of_word:
            sugestoes.append(prefixo)
        for char in sorted(nodo.children.keys()):
            self.listar_palavras(nodo.children[char], prefixo + char, sugestoes)

    def autocompletar(self, prefixo: str) -> List[str]:
        """Retorna uma lista de sugestões baseadas em um prefixo."""
        prefixo = prefixo.lower()
        nodo_prefixo = self.buscar_prefixo(prefixo)
        if not nodo_prefixo:
```

```
return []
sugestoes: List[str] = []
self.listar_palavras(nodo_prefixo, prefixo, sugestoes)
return sugestoes
```

```
# --- Programa Principal ---
```

```
def main():
    trie = Trie()
```

```
# Localiza o caminho absoluto do dicionário baseado na posição do script
diretorio_atual = os.path.dirname(os.path.abspath(__file__))
caminho_dicionario = os.path.join(diretorio_atual, 'dicionario.txt')
```

```
try:
    # Tenta carregar o dicionário de um arquivo conforme o enunciado
    with open(caminho_dicionario, 'r', encoding='utf-8') as f:
        # Usar set() remove duplicatas automaticamente antes da inserção
        palavras = {linha.strip() for linha in f if linha.strip()}
    for palavra in sorted(palavras):
        trie.inserir(palavra)
    print("Dicionário carregado do arquivo 'dicionario.txt'.")
except FileNotFoundError:
    # Fallback: Exemplo de dicionário padrão se o arquivo não existir
    dicionario = ["abacaxi", "abacate", "abóbora", "abobrinha", "amora", "banana", "batata"]
    for palavra in dicionario:
        trie.inserir(palavra)
    print("Aviso: Arquivo 'dicionario.txt' não encontrado. Usando lista padrão.")
```

```
print("\n> minha_frutaria")
```

```
while True:
    entrada = input("\nEscolha uma fruta ou legume: ").strip().lower()
```

```
if entrada == 'sair':
    print("Encerrando programa...")
    break
```

```
if not entrada:
    continue
```

```
sugestoes = trie.autocompletar(entrada)
```

```
if sugestoes:
    print("Sugestões:")
    for s in sugestoes:
        print(s)
else:
    print("Nenhuma sugestão encontrada.")
```

```
if __name__ == "__main__":
    main()
```

ARQUIVO TXT

```
dicionario = ["abacaxi", "abacate", "abóbora", "abobrinha", "amora", "banana", "batata"]
for palavra in dicionario:
    trie.inserir(palavra)

print("--- Minha Frutaria ---")
entrada = input("Escolha uma fruta ou legume: ").strip()

sugestoes = trie.autocompletar(entrada)

if sugestoes:
    print("\nSugestões:")
    for s in sugestoes:
        print(s)
else:
    print("\nNenhuma sugestão encontrada.")

if __name__ == "__main__":
    main()
```

XX

Sistema de Autocompletar com Árvore Trie

Este projeto implementa um algoritmo de autocompletar eficiente utilizando a estrutura de dados ****Trie**** (também conhecida como Árvore de Prefixos). O objetivo é sugerir palavras de um dicionário a partir de um prefixo digitado pelo usuário.

🚀 Como Executar

Certifique-se de estar na pasta do projeto e execute:

```
```bash
python3 termo_solver.py
```
```

O programa tentará carregar palavras de um arquivo chamado `dicionario.txt`. Caso não o encontre, utilizará uma lista padrão de frutas e legumes.

🧠 O que é uma Árvore Trie?

Diferente de uma lista linear onde precisaríamos percorrer todas as palavras para encontrar correspondências, a Trie organiza as strings em uma hierarquia de caracteres.

- ****Compartilhamento de Prefixos:**** Palavras como "abacaxi" e "abacate" compartilham os mesmos nós iniciais (`a` -> `b` -> `a`).

- ****Performance:**** A busca por um prefixo leva tempo **$O(m)$** , onde m é o comprimento do prefixo, independentemente de haver 10 ou 10.000 palavras no dicionário.

🛠️ Estrutura do Código

O arquivo `termo\_solver.py` está dividido em três partes principais:

1. Classe `TrieNode`

Representa cada "letra" na árvore.

- `children`: Um dicionário Python que mapeia o próximo caractere para o próximo nó. Usar um dicionário permite que a árvore suporte qualquer caractere (acentos, símbolos, etc).

- `is_end_of_word`: Um booleano que marca se aquele nó encerra uma palavra completa.

2. Classe `Trie`

Contém a lógica de manipulação da árvore:

- `inserir(palavra)`: Percorre a árvore caractere por caractere, criando novos nós se necessário, e marca o último nó como fim de palavra.

- `buscar_prefixo(prefixo)`: Navega pela árvore seguindo os caracteres do prefixo. Se o caminho for interrompido, o prefixo não existe.

- `listar_palavras(nodo, prefixo, sugestoes)`: Utiliza **Busca em Profundidade (DFS)** para encontrar todas as terminações possíveis a partir de um ponto da árvore.

- `autocompletar(prefixo)`: O método principal que une as funções anteriores para retornar a lista de sugestões.

3. Programa Principal (`main`)

Gerencia a interface com o usuário e a persistência de dados:

- ****Carregamento Robusto:**** Utiliza `os.path` para garantir que o arquivo `dicionario.txt` seja encontrado mesmo que o script seja chamado de diretórios diferentes.

- ****Limpeza de Dados:**** Usa um `set` para remover duplicatas e `strip()` para limpar espaços em branco ao ler o dicionário.

- ****Loop de Interação:**** Mantém o programa rodando para múltiplas consultas até que o usuário digite "sair".

📊 Complexidade Algorítmica

| Operação | Complexidade |
|------------------|---|
| --- | --- |
| Inserção | $O(L)$, onde L é o tamanho da palavra |
| Busca de Prefixo | $O(P)$, onde P é o tamanho do prefixo |
| Espaço | $O(N * L)$, onde N é o número de palavras (otimizado pelo reuso de prefixos) |

Trabalho Prático desenvolvido para a disciplina de Algoritmo